Lab Computational Analytics Report

Implementing and Evalutating KADABRA against BRAVA-GNN in Julia

Martin Schlaier

7th July 2026

Summer Semester

First Examiner:
Dr. Aurora Rossi

Second Examiner:

Department of Computer Science
University of Bonn

Declaration of generative AI and AI-assited technologies in the writing process

I hereby declare that generative artificial intelligence tools in the form of, `Gemini 3.5 Flash`, `Gemini 3.5 Pro` were used in the writing process. These tools were solely used to improve language and readability of this work. I edited and reviewed all the generated and modified content as needed and take full responsibility for the content in this work.

Contents

1 Introduction

1.1 Motivation

Identifying highly influential structures within large networks is a fundamental problem in graph analytics. The **Betweenness Centrality Problem (BCP)** formalizes this challenge by calculating the betweenness centrality for the nodes of a graph. This metric quantifies the importance of a node based on the number of shortest paths that pass through it.

Solving the BCP is critical for a wide range of real-world applications. These include identifying critical vulnerabilities in power grids, optimizing data traffic in telecommunication networks, mitigating urban traffic congestion, and securing global supply chains against systemic bottlenecks.

Exact algorithms for the BCP traditionally rely on modern shortest-path frameworks, most notably Brandes' algorithm (Brandes 2001), which executes multiple graph traversals (such as Dijkstra's algorithm). Although these exact methods guarantee optimal solutions, their computational runtime becomes prohibitive on large-scale graphs. Consequently, approximation methods such as **KADABRA** have been developed, successfully trading strict optimality for significantly improved efficiency (Borassi et al. 2019).

Recent advances in Graph Neural Networks (GNNs) show great promise. Specifically, the creators of BRAVA-GNN ('Degree-Mass Message Passing for Betweenness Ranking in Directed and Undirected Networks' 2026) have achieved state-of-the-art performance among GNN approaches, even outperforming KADABRA in full-graph evaluations. However, existing benchmarks primarily compare total graph calculations. A primary advantage of KADABRA is its ability to approximate betweenness centrality exclusively for the top-k nodes, offering substantial speedups by bypassing the rest of the network. This report extends the comparative analysis between KADABRA and BRAVA-GNN by focusing specifically on this top-k approximation performance. Furthermore, this report serves as supplementary documentation for the implementation and integration of the KADABRA algorithm into the Julia `Graphs.jl` ecosystem, as well as comparing the authors C++ implementation vs this new Julia.

1.2 Related Work

The development of polynomial-time exact algorithms for DSP has been a significant area of research. **picard_network_1982** published the first exact algorithm, which was fundamentally built on maximum-flow computations. This was later advanced by **goldberg_finding_1984**, who proposed a version with improved time complexity. A

different approach was taken by **charikar_greedy_2000**, who designed an exact algorithm based on a linear program (LP). The existence of an LP-based algorithm is not surprising, as the max-flow problem can be formulated as an LP. However, Charikar's method offers a new perspective, allowing his greedy-peeling approximation algorithm to be understood as a primal-dual algorithm for DSP. This greedy algorithm, which iteratively removes nodes with the lowest degree, provides a 2-approximation.

Building upon this concept, **iterative peeling algorithms** have been developed. Instead of performing the peeling process once using node degree as the criterion, it is executed multiple times. In each new iteration, more complex node weights, influenced by the results of the previous iterations, are used instead of the simple degree. **boob_flowless_2020** introduced the first algorithm of this type, called *Greedy++*, and demonstrated that it achieves a near-optimal result in just a few iterations and is an order of magnitude faster than Goldberg's maximum flow algorithm. **chekuri_densest_2022** showed that Greedy++ is an approximation scheme that can achieve a value arbitrarily close to the optimum given enough iterations, although the required number of iterations can be extremely high in theory.

hochbaum_flow_2024 introduced the *Incremental Parametric Cut* procedure (IPC), which uses the pseudoflow algorithm**hochbaum_pseudoflow_2008**. This approach uses previous results for initialization when the density parameter changes monotonically, thereby avoiding wasteful recomputations. Techniques like **restartable maximum flow** allow an existing flow solution to be efficiently updated after minor changes to the graph, instead of restarting the algorithm from scratch. **hochbaum_flow_2024** claim that their algorithm is faster by several orders of magnitude than *Greedy++*. However, the publication cited to support this claim is not available online.

Recent advances have produced new parametric flow algorithms with unique capabilities. The Parametric Breadth-First Search (PBFS) (**beines_simpler_2024**) and Parametric Pseudo Flow (PPF) (**Sauer_PPF_2025**) are the first algorithms designed to find a problem's breakpoints—critical parameter values where the minimum cut's structure changes—in strictly monotone order. This specific property opens up new strategies for solving ratio problems like the DSP.

The optimal density for the DSP corresponds to the breakpoint of the highest value. The existence of algorithms that find breakpoints in order enables a new solution strategy: if the problem can be reformulated such that this highest-value breakpoint becomes the *first* to be found, the solution process could be significantly accelerated.

1.3 Our Contribution

A central contribution of this work is the first systematic, empirical comparative study of modern parametric maximum-flow algorithms for the densest subgraph problem (DSP) versus established approximation methods. The investigation covers 47 real-world graphs and includes the first comprehensive and publicly documented evaluation of the Incremental Parametric Cut procedure (IPC) by **hochbaum_flow_2024**, given that their own empirical study is not publicly available. To enable this evaluation we implement

and assess two novel exact algorithms:

DSP-FBA, which leverages recent monotone parametric flow algorithms to find the solution directly by identifying the first breakpoint in a reformulated network, making it the only one required.

DSP-Intersect, an intersection-based algorithm using restartable maximum flow with both push-relabel and pseudoflow implementations, a similar approach to IPC.

Our experiments reveal that the choice of the best algorithm strongly depends on graph properties, particularly degree variance. For graphs with low variance, **DSP-FBA** performs best, whereas for graphs with high variance, IPC is superior. Confirming the hypothesis previously stated by Hochbaum (2024), our results challenge the common assumption that approximation algorithms are always preferable for large graphs. We demonstrate that exact algorithms can achieve optimal solutions with runtimes only slightly higher than those of approximations like Greedy++, making them a viable alternative when solution quality is critical.

Additionally, we systematically analyze the effectiveness of graph pruning based on the greedy algorithm by Charikar (2000). We show that pruning offers substantial benefits, especially for unweighted graphs with high degree variance, and support this with more extensive experiments. Specifically, we investigate the combination of pruning followed by these fast flow algorithms—a configuration not previously analyzed. We show that with pruning, exact algorithms on unweighted graphs can consistently outperform the unpruned Greedy++ approximation, while the benefit is far less pronounced for weighted graphs.

Our implementation extends the parametric maximum flow framework by **beines_simpler_2024** with new, DSP-specific algorithms. These contributions provide both theoretical insights into the performance of exact versus approximate algorithms and practical guidance for practitioners.

1.4 Betweenness Ranking Approximation

The betweenness centrality of a node measures its importance as a bridge or bottleneck based on the fraction of all shortest paths passing through it (**freeman1977set**; Brandes 2001). Given an unweighted graph $G = (V, E)$, let $\sigma_{s,t}$ denote the total number of shortest paths from node s to node t, and let $\sigma_{s,t}(v)$ be the number of those paths that pass through $v \in V$. The exact betweenness centrality $C_B(v)$ of a vertex v is formally defined as:

$$C_B(v) = \sum_{s \neq v \neq t \in V} \frac{\sigma_{s,t}(v)}{\sigma_{s,t}} \tag{1.1}$$

where $\frac{\sigma_{s,t}(v)}{\sigma_{s,t}} = 0$ if $\sigma_{s,t} = 0$ (**brandes2001faster**). In literature and practical network libraries, normalized variants are commonly used to ensure scale-invariance across different graph sizes.

The standard normalization factor depends on whether the source and target node pairs can include v in their normalization pool. For instance, the Julia `Graphs.jl` library normalizes by dividing by the total number of pairs excluding v:

$$C_B(v) = \frac{1}{(n-1)(n-2)} \sum_{s \neq v \neq t \in V} \frac{\sigma_{s,t}(v)}{\sigma_{s,t}} \tag{1.2}$$

Alternatively, the adaptive approximation algorithm KADABRA treats betweenness centrality probabilistically (Borassi et al. 2019). It frames the metric as the exact probability that a shortest path π between two uniformly selected random nodes s and t passes through v (**borassi2019kadabra**). Following the convention where the sum includes cases where $\sigma_{s,t}(v) = 0$ whenever $s = v$ or $t = v$, the normalized centrality is expressed as:

$$C_B(v) = \frac{1}{n(n-1)} \sum_{s \neq t \in V} \frac{\sigma_{s,t}(v)}{\sigma_{s,t}} \tag{1.3}$$

1.4.1 Betweenness Ranking Approximation Problem

Given the prohibitive $O(|V||E|)$ exact computation cost of Brandes' algorithm on massive networks, the goal of the *Betweenness Ranking Approximation Problem* is to learn a size-invariant function $f : V \to \mathbb{R}$ that maps each node to a real-valued score $f(v)$ such that the induced relative ordering maximizes the ordinal correlation with the true centrality hierarchy:

$$\max_{f} \tau \left(\mathcal{R}(f(V)), \mathcal{R}(C_B(V)) \right) \tag{1.4}$$

Where $\mathcal{R}(\cdot)$ represents the rank operator, and τ denotes a ranking index such as the Kendall Tau correlation coefficient.

This formulation allows the estimation to be generalized natively across edge-weighted or directed graph structures. For directed representations, the flow maps independent path dynamics from incoming and outgoing topologies before scoring fusion ($w_{u,v} = 1$ for structural baselines).

For later structural representation, we define the m-th order multi-hop degree mass of a node vector as:

$$d^{(m)} = \left(\sum_{k=0}^{m} A^k \right) d \tag{1.5}$$

as the cumulative structural neighborhood volume within m hops, where A represents the graph adjacency matrix and d is the standard degree vector. In strictly local configurations, $m = 1$ holds.

The problem is equivalent to preserving topological dominance hierarchies without minimizing absolute regression errors. For high-diameter configurations like spatial road networks, structural regularities produce strict discrete symmetry boundaries, causing standard scale-free embeddings to experience over-smoothing. By shifting the inputs to

size-invariant degree masses, long-range structural bottlenecks are captured efficiently within a lightweight parameter footprint.

$$\mathcal{L}(s_u, s_v, y) = \max(0, 1 - y \cdot (s_u - s_v)) \tag{1.6}$$

Since exact deep architectures for this structural proxy require representative generative training, formulations leverage synthetic hyperplanes such as the hyperbolic random graph model to calibrate power-law degree variations outside standard scale-free assumptions prior to message passing.

2 KADABRA

2.1 Overview and Problem Formalization

While exact algorithms like Brandes' compute Betweenness Centrality (BC) for all nodes in $O(mn)$ time, this worst-case complexity becomes prohibitive for large-scale real-world networks. KADABRA shifts the paradigm from exact computation to sampling-based approximation.

2.1.1 The Stochastic Paradigm

The core premise relies on a probabilistic interpretation of betweenness centrality. If we select a source node s and a target node t uniformly at random, and subsequently choose a single shortest path π between them uniformly at random from all available st-shortest paths, the betweenness centrality $bc(v)$ of a node v is exactly equal to the probability that π passes through v:

$$\Pr(v \in \pi) = bc(v) \tag{2.1}$$

KADABRA leverages this by sampling a sequence of τ random shortest paths $\pi_1, \ldots, \pi_\tau$ according to this exact distribution. For each sampled path, an empirical indicator variable $X_i(v)$ is recorded:

$$X_i(v) = \begin{cases} 1 & \text{if } v \in \pi_i \text{ and } v \neq s, t \\ 0 & \text{otherwise} \end{cases} \tag{2.2}$$

The algorithm's empirical estimate of betweenness centrality, denoted as $\tilde{b}(v)$, is simply the sample mean of these indicators:

$$\tilde{b}(v) := \frac{1}{\tau} \sum_{i=1}^{\tau} X_i(v) \tag{2.3}$$

By definition, this estimator is unbiased, yielding an expected value $\mathbb{E}[\tilde{b}(v)] = bc(v)$.

2.1.2 Objective and Error Formalization

The objective of KADABRA is to minimize the required number of sampled paths τ while providing strict statistical guarantees on the quality of the approximation across all vertices. The user specifies two global tolerance parameters:

- An **absolute error bound** $\lambda > 0$.

- A **confidence parameter** $\delta \in (0, 1)$.

The standard problem task is to ensure that, upon termination, the empirical estimates for all nodes fall within a $1-\delta$ simultaneous confidence interval. Formally, the algorithm guarantees that:

$$\Pr(\forall v \in V, |\tilde{b}(v) - bc(v)| \leq \lambda) \geq 1 - \delta \tag{2.4}$$

2.1.3 The Adaptive Sampling Challenge

A naïve approximation algorithm fixes a static sample size τ in advance using worst-case bounds derived from standard concentration inequalities (such as Hoeffding's or Chernoff's bounds). However, this uniformly over-samples nodes with low centrality, which require significantly fewer paths to approximate accurately.

KADABRA optimizes this by utilizing **adaptive sampling**, meaning the total number of samples τ is treated as a dynamic random variable (a stopping time) determined on the fly based on the specific paths observed during execution.

The technical challenge—and a major point of departure from prior literature—is that evaluating a stopping condition based on the data introduces a subtle **stochastic dependence** between the termination time τ and the correctness of the estimator. KADABRA resolves this by using martingale theory to ensure that checking bounds dynamically does not compromise the mathematical strictness of the (λ, δ)-guarantee.

2.2 Algorithmic Innovations in KADABRA

KADABRA achieves its massive speedup over previous state-of-the-art approximation algorithms by optimizing both the time required to generate an individual path sample and the total number of samples required to guarantee precision.

2.2.1 Balanced Bidirectional BFS (bb-BFS)

Most existing betweenness approximation algorithms generate a sample path by picking a random source node s and executing a standard single-source Breadth-First Search (BFS) to explore the graph. In the worst-case, this demands $\Theta(|E|)$ time per sample.

KADABRA introduces a **balanced bidirectional BFS** (bb-BFS) routine to optimize path generation. Given a random source s and target t, the algorithm simultaneously grows a forward BFS tree from s and a backward BFS tree from t. To ensure the search remains computationally balanced, the algorithm keeps track of the boundary volumes:

- Let $\Gamma^{l_s}(s)$ denote the set of vertices at distance l_s from s.
- Let $\Gamma^{l_t}(t)$ denote the set of vertices at distance l_t from t.

At each step, the algorithm chooses to expand from the tree with the smaller boundary volume, minimizing the immediate edge-traversal cost:

$$\text{Expand from } s \iff \sum_{v \in \Gamma^{l_s}(s)} \deg(v) \leq \sum_{u \in \Gamma^{l_t}(t)} \deg(u) \tag{2.5}$$

The bidirectional search terminates as soon as the boundaries of the two trees "touch each other" (i.e., a common edge or node is discovered). A valid st-shortest path is then constructed by connecting a uniformly chosen random path from s to the meeting boundary, and tracking back from the boundary to t.

Crucially, the authors prove that on realistic random complex networks (such as Configuration Models or Rank-1 Inhomogeneous Random Graphs), bb-BFS cuts down path generation time from worst-case linear time $\Theta(|E|)$ to $|E|^{\frac{1}{2}+o(1)}$ with high probability.

2.2.2 Rigorous Adaptive Stopping Condition

To minimize the number of samples τ, KADABRA evaluates a stopping condition dynamically on the fly. Rather than tracking error from a predefined global count, KADABRA sets up an absolute maximum sample limit ω (derived from worst-case VC-dimension bounds) and computes deterministic target allocation confidences $(\delta_L^{(v)}, \delta_U^{(v)})$ for each node v during an initial brief profiling phase.

At each step τ, the algorithm evaluates lower and upper error margins (f and g) based on the empirical centrality computed up to that point. The mathematical bounds are derived as:

$$f(\tilde{b}(v), \delta_L^{(v)}, \omega, \tau) = \frac{1}{\tau} \ln \frac{1}{\delta_L^{(v)}} \left(\frac{1}{3} - \frac{\omega}{\tau} + \sqrt{\left(\frac{1}{3} - \frac{\omega}{\tau} \right)^2 + \frac{2\tilde{b}(v)\omega}{\ln(1/\delta_L^{(v)})}} \right) \tag{2.6}$$

$$g(\tilde{b}(v), \delta_U^{(v)}, \omega, \tau) = \frac{1}{\tau} \ln \frac{1}{\delta_U^{(v)}} \left(\frac{1}{3} + \frac{\omega}{\tau} + \sqrt{\left(\frac{1}{3} + \frac{\omega}{\tau} \right)^2 + \frac{2\tilde{b}(v)\omega}{\ln(1/\delta_U^{(v)})}} \right) \tag{2.7}$$

The sampling loop dynamically terminates as soon as the loop condition maps to `true`, checking if $f \leq \lambda$ and $g \leq \lambda$ for all relevant vertices. By framing the sequence of sample path indicators as a martingale, the algorithm ensures that evaluating this stopping rule dynamically over a stochastic running time τ does not break the target $(1-\delta)$ probability guarantee.

2.3 Theoretical Guarantees: The Contrast with GNNs

The foundational justification for employing KADABRA over other modern approximation methodologies lies in its rigorous mathematical guarantees. When evaluating a sampling-based approach against learning-based models like Graph Neural Networks (GNNs), this section highlights the trade-offs between provable statistical bounds and inductive heuristics.

2.3.1 Rigorous Martingale-Based Correctness

The core mathematical achievement of KADABRA is proving that its dynamic stopping condition preserves strict probabilistic safety. In standard adaptive sampling, terminating

an algorithm based on the observed data introduces a stochastic dependence that can bias the final estimates.

By modeling the sequence of path indicator variables as a martingale Z^τ and applying McDiarmid's martingale concentration inequality, the authors prove that KADABRA delivers an absolute (λ, δ)-approximation upon termination:

$$\Pr\left(\forall v \in V,\, |\tilde{b}(v) - bc(v)| \leq \lambda\right) \geq 1 - \delta \tag{2.8}$$

Whether the algorithm stops early ($\tau < \omega$) via its adaptive triggers, or reaches the worst-case sample bound (ω), the mathematical certainty of the confidence interval remains completely unbroken.

2.3.2 Adaptive Top-k Centrality Selection

A significant limitation of previous betweenness approximation frameworks is their architectural rigidity: they require computing a uniform λ-approximation across *all* nodes to construct a reliable top-k ranking. This introduces massive computational overhead, as identical precision is wasted on low-centrality nodes that are mathematically far from the top-k boundary. KADABRA resolves this by decoupling individual node confidence intervals, allowing larger margins of error for nodes whose centrality values are structurally well-separated.

Given a user-defined threshold k and absolute tolerance λ, KADABRA guarantees that upon termination, for every vertex v, the algorithm either:

1. Establishes the exact correct positional rank of v.
2. Formally guarantees that $v \notin$ Top-k.
3. Provides an empirical estimate such that $|bc(v) - \tilde{b}(v)| \leq \lambda$.

To implement this on the fly, KADABRA replaces the global, uniform checking condition with a dynamic boundary validation filter. At each sampling iteration τ, the vertices are sorted in descending order of their empirical centrality estimates to form a tentative sequence $v_1, v_2, \ldots, v_n$. The algorithm is allowed to safely terminate early if and only if the following strict separation criteria are simultaneously met:

- **Internal Top-k Boundary Separation:** For every internal index $i \in [1, k]$, if the bounds $f(v_i)$ or $g(v_i)$ still exceed λ, the algorithm checks if their empirical confidence intervals overlap with their immediate neighbors:

$$\tilde{b}(v_{i-1}) - f(v_{i-1}) \geq \tilde{b}(v_i) + g(v_i) \quad \text{and} \quad \tilde{b}(v_i) - f(v_i) \geq \tilde{b}(v_{i+1}) + g(v_{i+1}) \tag{2.9}$$

  If these lower and upper bounds cross, the true positions are not yet statistically distinct, forcing the algorithm to sample more paths.

- **External Boundary Exclusion:** For every node outside the top ranking $i \in [k+1, n]$, if the bounds $f(v_i)$ or $g(v_i)$ exceed λ, the algorithm ensures that the node cannot breach the Top-k barrier:

$$\tilde{b}(v_k) - f(v_k) \geq \tilde{b}(v_i) + g(v_i) \tag{2.10}$$

  This strictly confirms that v_i cannot have a true betweenness centrality higher than the k-th most central node.

This adaptive relaxation allows KADABRA to skip massive volumes of unnecessary shortest-path sampling. In practice, it enables the first successful approximation of top-k centralities on graphs featuring hundreds of millions of edges (such as massive citation or collaboration networks) in less than an hour.

3 brava

4 Experimental Setup

4.1 Hardware

The experiments were conducted on a server equipped with an **AMD Ryzen Threadripper 3960X 24-Core Processor** clocked at **3.80 GHz**. The processor features **24 cores**, each with **2 threads**, and is supported by **125,GiB of RAM**. Cache specifications include **768,KiB L1d**, **768,KiB L1i**, **12,MiB L2**, and **128,MiB L3**. The system is additionally equipped with two **NVIDIA GeForce RTX 2080 Ti GPUs**, each with **11,GiB of VRAM**. The operating system used was **Ubuntu 22.04 LTS**.

The original project by Borassi et al. (2019) was implemented in **C++** and we compiled using **CMake version 4.2.1** with **GNU 11.4.0**. Our implementation of KADABRA and BRAVA-GNN was implemented and run with **Julia 1.12.5**. **Simexpal 1.1** (**simexpal2019**) was used to conduct the experiments.

4.2 Implementation

As basis for the implementation serves the framework for parametric maximum flow created by **beines_simpler_2024** in which we included the implementation of pseudoflow and PPF by **Sauer_PPF_2025**. Using these implementation of push relabel, pseudoflow, PBFS and PPF we implemented the Algorithms for DSP as described above. For Greedy++ we based it on the implementation by **boob_flowless_2020**. For a fairer comparison we adapted this implementation to use the same C++ datastructures and encapsulate it in a class, as we did with the flow based algorithms. Since constructing the flow network (Section **??**) plays an important role in the runtime for parametric maximum flow, we therefore must also include the initiation of the necessary data structures for Greedy++.

4.3 Instances

For comparability, we use the same instances as 'Degree-Mass Message Passing for Betweenness Ranking in Directed and Undirected Networks' (2026). A variety of instances are obtained from the SNAP database (**leskovec_snap_2014**), ASU's Social Computing Data Repository (**zafarani_social_2009**), and the Koblenz Network Collection (**kunegis_konect_2013**).

All unweighted instances used in our experiments are listed in Table **??**

4.4 Algorithm Performance Metrics

We conducted a detailed performance analysis of each exact algorithm on every graph instance. For the exact methods, we recorded two primary metrics:

1. the time required to construct the flow network,
2. the runtime of the algorithm.

For *Greedy++*, we evaluated its performance as an approximation algorithm. Given the optimal solution value, we specified target percentages $(0.80, 0.90, 0.95, 0.99, 1.00)$ and measured both runtime and number of iterations until the algorithm achieved at least each target fraction of the optimum. Following **boob_flowless_2020** and confirmed by our experiments that three iterations always reach at least 90% of the optimum for the unweighted instances, we also executed Greedy++ with exactly three iterations and recorded its runtime.

All experiments were repeated on both the original and the *pruned* graphs (see Chapter **??**). For pruning we tracked the pruning time and the resulting graph size.

Each algorithm–instance pair was run five times with a 30-minute time limit; we report the median of these runs. For pruning statistics we used the median of all instances and runs.

Throughout this thesis we use the following shorthand:

- **PF**: IPC or DSP-Intersect using pseudoflow, depending on context.
- **PR**: IPC or DSP-Intersect using push–relabel, depending on context.
- **FPPF**: DSP-FBA using parametric pseudoflow.
- **FPBFS**: DSP-FBA using parametric breadth-first search.
- **G-XX**: Greedy++ run until XX% of the optimum is reached (e.g., G-90).
- **G-3**: Greedy++ run for exactly three iterations.
- **P-X**: Pruning Version.

4.5 Results

instance	n	m	m/n
p2p-Gnutella30	36 682	88 328	2.41
email-Enron	36 692	367 662	10.02
musae-github	37 700	289 003	7.67
gemsec-Facebook	50 515	819 306	16.22
p2p-Gnutella31	62 586	147 892	2.36
soc-Epinions1	75 879	508 837	6.71
soc-Slashdot0811	77 360	905 468	11.70
soc-Slashdot0902	82 168	948 464	11.54
twitch-gamers	168 114	6 797 557	40.43
email-EuAll	265 214	420 045	1.58
com-DBLP	317 080	1 049 866	3.31
web-NotreDame	325 729	1 497 134	4.60
web-BerkStan	685 230	7 600 595	11.09
web-Google	875 713	5 105 039	5.83
com-youtube	1 134 890	2 987 624	2.63
soc-Pokec	1 632 803	30 622 564	18.75
wiki-topcats	1 791 489	28 511 807	15.92
amazon	2 146 057	5 743 146	2.68
wiki-Talk	2 394 385	5 021 410	2.10
com-Orkut	3 072 441	117 185 083	38.14
cit-Patents	3 764 117	16 511 741	4.39
com-lj	3 997 962	34 681 189	8.67
dblp	4 000 148	8 649 011	2.16
soc-LiveJournal1	4 847 571	68 993 773	14.23

Table 4.1: Overview of instances: n is the number of vertices, m the number of edges, Sources: [1] KONECT (**kunegis_konect_2013**), [2] Zafrani (**zafarani_social_2009**); unmarked instances are from SNAP (**leskovec_snap_2014**).

5 Conclusion

A Absolute Runtimes

Bibliography

Borassi, Michele and Emanuele Natale (20th Feb. 2019). 'KADABRA is an ADaptive Algorithm for Betweenness via Random Approximation'. In: *ACM J. Exp. Algorithmics* 24, 1.2:1–1.2:35. ISSN: 1084-6654. DOI: 10.1145/3284359. URL: https://dl.acm.org/doi/10.1145/3284359 (visited on 13/04/2026).

Brandes, Ulrik (June 2001). 'A faster algorithm for betweenness centrality*'. In: *The Journal of Mathematical Sociology* 25.2, pp. 163–177. ISSN: 0022-250X, 1545-5874. DOI: 10.1080/0022250X.2001.9990249. URL: http://www.tandfonline.com/doi/abs/10.1080/0022250X.2001.9990249 (visited on 13/04/2026).

'Degree-Mass Message Passing for Betweenness Ranking in Directed and Undirected Networks' (2026). In.